

人工智能基础与应用 · 大模型实训

M0 · 数学与机器学习地基

从零实现大模型：M0-M9 知识主干（16 周 / 11 课题 / 零预算）

孙承泽 · 能动强基 2501 · 西安交通大学

2026-09-17 生成 · 版本 v1.0

Markdown 母版：人工智能基础与应用/讲义/ 下的 .md 文件 · 配套上游：CS336 A1-A5 · nanochat · minimind

讲义 M0 · 数学与机器学习地基

对应：贯穿全程，W1 起 | 前置讲义，不单独占周次，随用随补 对标材料：李沐 d2l ch2–ch5 + ch11（优化）· Karpathy micrograd 视频 · probability/ 课题 01–08（已建成，直接搭桥）为什么先讲这个：后面所有的“为什么训练崩了”“为什么显存不够”“为什么换分词器 loss 不能比”，答案都在这一讲里。它不是复习，是工具箱。

〇、本讲要回答的 5 个问题

1. 梯度是怎么穿过 100 层网络的？为什么反向传播只要一遍？
2. 交叉熵、困惑度、bpb 三者的关系是什么？为什么跨模型只能比 bpb？
3. 训练一个 N 参数的模型，显存和算力到底怎么估？
4. 为什么 softmax 要先减最大值？不这样做会怎样？
5. “loss 不降”时，我该按什么顺序怀疑？

一、自动微分：为什么反向传播只要一遍

计算图：把 $f(x_1, \dots, x_n)$ 表示为有向无环图，每个节点是一次基本运算。链式法则：若 $y = f(g(x))$ ，则 $\frac{dy}{dx} = \frac{dy}{dg} \cdot \frac{dg}{dx}$ 。

两种方向的差别（这是本讲的第一个关键直觉）：

模式	做法	成本	适合
前向模式	从输入往输出推导数	每个输入要一遍 → n 遍	输入少、输出多
反向模式	从输出往输入推导数	一遍拿到全部输入的偏导	输入多、输出少 ← 我们的情况

我们的情况是“几亿个参数（输入多）→ 一个标量 loss（输出少）”，所以反向模式（就是我们说的反向传播）是**首选**：1 次前向 + 1 次反向 = 全部梯度。

梯度累加的工程含义：反向时每个节点把上游梯度**相加**。这解释了两个常见现象：①同一个张量被用了两次（如权重共享），梯度会累加；②既然梯度是“累加”的，就能**梯度累积**（gradient accumulation）——多个小批次的梯度加起来等价于大批次，这是后来显存不够时的核心手段（课题 04 会用到）。

在代码里长什么样：

- 手写/里会有一个 Value 类（或等价的张量自动微分），核心是 `.backward()`：按**拓扑逆序**遍历，每个节点用局部导数乘上游梯度累加到父节点。
- 上游对照：nanocat/optim.py 里参数更新前的 `p.grad` 就是这一机制的产物；CS336 A1 的 `run_gradient_clipping` 则是在使用梯度（先要能算出来）。

二、信息量、交叉熵、困惑度、bpb（跨分词器可比性的根源）

信息量：一个概率为 p 的事件携带的信息是 $-\log p$ （nats，底为 e ；若底为 2 则单位是 bit）。

熵：随机变量的平均信息量 $H(p) = -\sum_x p(x) \log p(x)$ 。直觉：“平均每个符号有多难猜”。

交叉熵：用模型分布 q 去编码真实分布 p 的代价

$$H(p, q) = -\sum_x p(x) \log q(x)$$

语言模型里 p 是“真实下一个 token”， q 是模型输出——这正好就是训练 loss。

困惑度： $\text{PPL} = \exp(H(p, q))$ (nats 为底时)。直觉最有用的一句话：“模型平均在多少个等概率选项里犯迷糊。” $\text{PPL}=10$ 相当于“平均 10 选 1”。

bpb (bits per byte)：

$$\text{bpb} = \frac{H(p, q) [\text{nats}]}{\ln 2 \times \# \text{bytes per token}}$$

分母是“平均每个 token 对应多少字节”。

为什么必须用 bpb 跨模型比较：交叉熵的单位是“每 token”，而 token 的大小随分词器变化——词表越大 token 越“大”，同样文本的 token 数越少，单 token 交叉熵天然越高。除以“每 token 字节数”就把记账单位统一到“每个字节”上，于是与分词器无关。这也解释了 M2 讲义的结论：换词表后 loss 不可比，bpb 可比。

在代码里长什么样：nanochat/loss_eval.py 专门做 bpb（而不是 loss）的评估；nanochat/core_eval.py 则是另一类指标（CORE，由 22 个数据集的准确率合成）。你在课题 01 的压缩率表，其实是在预演 bpb 的分母——先感受“每 token 多少字节”。

三、算力与显存记账（估数量级是核心能力）

参数量（Transformer 主项，细节见 M3）：

- 每层注意力 $4d^2$ （Q/K/V/O 四个 $d \times d$ ），MLP 约 $8d^2$ （SwiGLU 的等价形式）→ 每层 $\approx 12d^2$
- 词嵌入 $V \cdot d$ （权重共享时 1 份）

算力（训练）：业界通用估算

$$\text{总 FLOPs} \approx 6 \times N \times D$$

其中 N 是非嵌入参数量， D 是训练 token 数。拆解：每个参数在前向贡献 2 FLOPs（乘+加），反向是前向的 2 倍 → 合计 6。注意力部分另有 $12Lhqt$ 项（序列长度相关，短序列时可忽略）。

Karpathy 在 nanochat 文档里给的就是这个公式，并特别注明“与 Chinchilla 略有 1% 差异，因为不计嵌入与 softmax”。

显存（训练）：粗算每参数

项	字节数
参数 (bf16)	2
梯度 (bf16)	2
Adam 一阶动量 (fp32)	4
Adam 二阶动量 (fp32)	4
小计（不含激活、不含 fp32 主权重）	$\approx 12-16$
激活（随 batch $\times$ 序列长度增长，可用 checkpointing 换）	视配置

练习式直觉：0.1B 参数 $\times 16$ 字节 $\approx 1.6\text{GB}$ （优化器+梯度+权重）；剩下的全是激活与临时张量——这就是为什么 16GB 的 T4 训 0.1B 要精打细算（课题 04 的真实约束）。

在代码里长什么样：nanochat/gpt.py 里有 estimate_flops() 与 num_scaling_params() 两个方法，正是上面两段公式的代码化。课题 03 的 scaling law 就是靠这两个函数把“跑多大”换算成“花多少算力”的。

四、数值稳定性：softmax 与 logsumexp

问题： $\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$ 。若某个 $x_i = 1000$ ， e^{1000} 直接溢出成 inf。

解法：分子分母同乘 e^{-m} ($m = \max_j x_j$):

$$\text{softmax}(x)_i = \frac{e^{x_i - m}}{\sum_j e^{x_j - m}}$$

数值上完全等价，但最大指数变成 $e^0 = 1$ ，永不溢出。“减最大值”不是技巧，是恒等变形。

logsumexp 技巧: $\log \sum_j e^{x_j} = m + \log \sum_j e^{x_j - m}$ ，这是所有“用 log 概率做加法”的算法（交叉熵、CTC、HMM、flash attention 的 online softmax）的共同基础。

为什么这很重要：课题 05 的 **online softmax** 就是“分块计算时如何边算边保持数值稳定”，其思想内核与这里的减最大值完全一致——你在 M0 学的东西，会在 Triton kernel 里原样再出现一次。

五、“loss 不降”的排查顺序（把知识变成动作）

1. **数据**：label 与输入错位了吗？分词是否正常（看打印出来的样本）？
2. **初始化**：全零/全同初始化会让对称性无法打破；检查 std 与层数是否匹配
3. **学习率**：太大 → loss 震荡/NaN；太小 → 曲线是平的。先用小数据能否过拟合一条样本自检
4. **形状与 mask**：mask 是否在 softmax 之前施加？是否广播到正确维度？
5. **数值**：是否出现 inf/nan？打开梯度范数监控
6. **数值精度**：bf16 在小模型/小梯度上可能不够（T4 没有 bf16，课题 04 要用 fp16 + GradScaler）

这份顺序本身就是“知行合一”：它将成为课题 04 的排查清单，而不是一段可背诵的文字。

六、能立刻跑的实验

```
# ① 手算 bpb 的换算（先自己推，再看输出对不对）
环境/.venv/bin/python - <<'PY'
import math
# 假设某模型：每 token 交叉熵 2.0 nats，平均每个 token 对应 4 个字节
loss_nats, bytes_per_token = 2.0, 4.0
ppl = math.exp(loss_nats)
bpb = loss_nats / math.log(2) / bytes_per_token
print(f"loss={loss_nats} nats → PPL={ppl:.2f}, bpb={bpb:.3f}")
print("→ 再推一遍：若词表翻倍使 bytes/token 变成 6，bpb 会怎么变？loss 呢？")
PY

# ② 验证“减最大值”是恒等变形（含极端值）
环境/.venv/bin/python - <<'PY'
import numpy as np
x = np.array([0.0, 1000.0, 1000.0])
naive = np.exp(x) / np.exp(x).sum()      # 溢出
stable = np.exp(x - x.max()) / np.exp(x - x.max()).sum()
print("naive :", naive, " ← 溢出成 nan")
print("stable:", stable, " ← 正确")
PY
```

七、代码级提问（3–5 道）

1. **【推导】** 用 e^{-m} 的恒等变形，证明“减最大值”不改变 softmax 的输出（写出两步）。

2. 【估算】 一个 0.1B 参数模型用 Adam 训练 (bf16 参数 + fp32 优化器状态): ①权重+梯度+优化器状态占多少 GB? ②若 T4 只有 16GB, 留给激活的预算是多少?
 3. 【预测】 训练中把 loss 从 fp32 计算改成 fp16 计算: 训练会更快还是更慢? 会出现什么新故障? (提示: T4 无 bf16; GradScaler 是干什么的)
 4. 【联系】 probability/ 里学过的“期望”和这里的“交叉熵”是同一个东西的两种说法吗? 试着用期望的符号把交叉熵写出来。
 5. 【诊断】 曲线在 400 步突然 NaN。按本讲 § 五 的顺序, 你会先看哪三个量?
-

八、记忆锚

- 隐喻: 反向传播 = “责任回溯”——loss 是最终事故, 每个参数按“它贡献了多少”分摊责任; 责任可以累加, 所以梯度能攒 (梯度累积)。
- 口诀: “一遍反向、六倍算力、十六字节、先减最大值” (反向传播一遍 / 6ND / 每参数约 16 字节 / softmax 稳定性)
- 符号位置法: 把 $\text{bpb} = \frac{H}{\ln 2 \cdot B}$ 写在卡片中央, 分子 H 是“每 token 的惊讶”, 分母 B 是“每 token 的字节”, 中间一条分数线 = “把单位从 token 换成 byte”——这条线就是跨分词器比较的桥。